

TASK - 7

(laptop acts as server using mqtt protocol)

TASK :

Esp32 is connected to ultrasonic sensor and the data is sent to laptop which acts as server using MQTT protocol and both esp and laptop are connected to same wifi (we can connect it to different wifi the only condition is that both the devices should have a stable network connection)

MQTT (Message Queuing Telemetry Transport)

MQTT (Message Queuing Telemetry Transport) is a lightweight network protocol designed to send data between devices. It is built specifically for constrained devices—like microcontrollers, sensors, and smart appliances—that operate on low power or low-bandwidth networks.

Instead of devices talking directly to each other, MQTT uses a Publish/Subscribe model.

- Publishers: Devices (like your ESP32) send data to a specific topic (e.g., sensor/ultrasonic).
- Subscribers: Devices (like your laptop) listen to that topic to receive the data automatically.

What is an MQTT Broker?

An MQTT Broker is the central server that manages all communication in an MQTT network. It acts as a mailman or a traffic controller. Devices do not send messages directly to other devices; instead, they send everything to the broker. The broker receives all incoming messages, filters them by their specific topic, and immediately forwards them to any devices that have subscribed to those topics. It handles authentication, ensures message delivery, and **prevents** devices from needing to know each other's IP addresses.

MQTT is lightweight and designed specifically for IoT devices.

Instead of installing an MQTT broker on a laptop we will use a **free, public cloud MQTT broker** provided by HiveMQ (broker.hivemq.com). This means the laptop and ESP32 just need internet access to talk to each other.

Step 1: Install the MQTT Library in VS Code

First, your laptop's Python environment needs a library to talk to the MQTT Broker.

1. Open **VS Code**.
2. Open a **New Terminal**.
3. Run the following command to install

```
paho-mqtt:  
pip install paho-mqtt
```

Step 2: Write the MQTT Subscriber Code in VS Code

Create a new file in VS Code named `mqtt_laptop.py` and paste the following code. This script will listen to the cloud broker channel.

```
import paho.mqtt.client as mqtt

# Configuration
MQTT_BROKER = "broker.hivemq.com"
MQTT_PORT = 1883
# Use a unique topic so other people on the public broker don't
interfere with your project
MQTT_TOPIC = "anagha_iot_project/distance"

# Callback function when the laptop successfully connects to the
broker
def on_connect(client, userdata, flags, rc):
    if rc == 0:
        print("[*] Successfully connected to HiveMQ Broker!")
        # Subscribe to our specific topic
        client.subscribe(MQTT_TOPIC)
        print(f"[*] Subscribed to topic: {MQTT_TOPIC}. Waiting for
ESP32 data...")
    else:
        print(f"[-] Connection failed with code {rc}")

# Callback function when a new message arrives from the ESP32
def on_message(client, userdata, msg):
    # Decode the message payload from bytes to string
    distance_data = msg.payload.decode("utf-8")
    print(f"MQTT Received - Distance: {distance_data} cm")

# Set up the MQTT Client
client = mqtt.Client()
client.on_connect = on_connect
client.on_message = on_message

# Connect to the public broker
print("[*] Connecting to broker...")
client.connect(MQTT_BROKER, MQTT_PORT, 60)

# Keep the script running in an infinite loop to listen for data
try:
    client.loop_forever()
except KeyboardInterrupt:
    print("\n[*] Disconnecting from broker. Goodbye!")
```

```
client.disconnect()
```

Step 3: Write the MQTT ESP32 Code in Thonny

MicroPython requires an external library named `umqtt.simple` to handle MQTT. Install it on ESP32

A. Install `umqtt.simple` on ESP32:

1. Make sure your ESP32 is plugged in and connected to Thonny.
2. Go to the top menu: **Tools > Manage Packages**.
3. Search for `umqtt.simple`.
4. Click on it and select **Install**. (Ensure your laptop has an active internet connection while doing this).

B. The MicroPython Code:

Create a new file in Thonny and paste this code. [Update Wi-Fi details](#)

```
import machine
import time
import network
from umqtt.simple import MQTTClient

# 1. Hardware Pin Setup
trigger = machine.Pin(5, machine.Pin.OUT)
echo = machine.Pin(18, machine.Pin.IN)

# 2. Network & MQTT Configuration
WIFI_SSID = "YOUR_WIFI_NAME"           # Replace with your Wi-Fi Name
WIFI_PASSWORD = "YOUR_WIFI_PASSWORD"  # Replace with your Wi-Fi Password

MQTT_BROKER = "broker.hivemq.com"
MQTT_PORT = 1883
MQTT_TOPIC = b"anagha_iot_project/distance" # Must match VS Code exactly (as bytes)
CLIENT_ID = "ESP32_Ultrasonic_Client"

def get_distance():
    trigger.value(0)
    time.sleep_us(2)
    trigger.value(1)
    time.sleep_us(10)
    trigger.value(0)
```

```

while echo.value() == 0:
    signaloff = time.ticks_us()
while echo.value() == 1:
    signalon = time.ticks_us()

timepassed = signalon - signaloff
distance = (timepassed * 0.0343) / 2
return round(distance, 2)

# Connect to Wi-Fi
wlan = network.WLAN(network.STA_IF)
wlan.active(True)
wlan.connect(WIFI_SSID, WIFI_PASSWORD)

print("Connecting to Wi-Fi...")
while not wlan.isconnected():
    time.sleep(1)
print("Connected to Wi-Fi! ESP32 IP:", wlan.ifconfig()[0])

# Connect to MQTT Broker
print("Connecting to MQTT Broker...")
try:
    client = MQTTClient(CLIENT_ID, MQTT_BROKER, port=MQTT_PORT)
    client.connect()
    print("Connected to HiveMQ Broker successfully!")
except Exception as e:
    print("Could not connect to MQTT Broker:", e)

# Main Loop
while True:
    try:
        dist = get_distance()
        print("Local reading:", dist, "cm")

        # Publish distance payload to the broker
        payload = str(dist)
        print(f"Publishing to {MQTT_TOPIC.decode()}: {payload}")
        client.publish(MQTT_TOPIC, payload)

    except Exception as e:
        print("Error sending data via MQTT:", e)

    time.sleep(2)

```

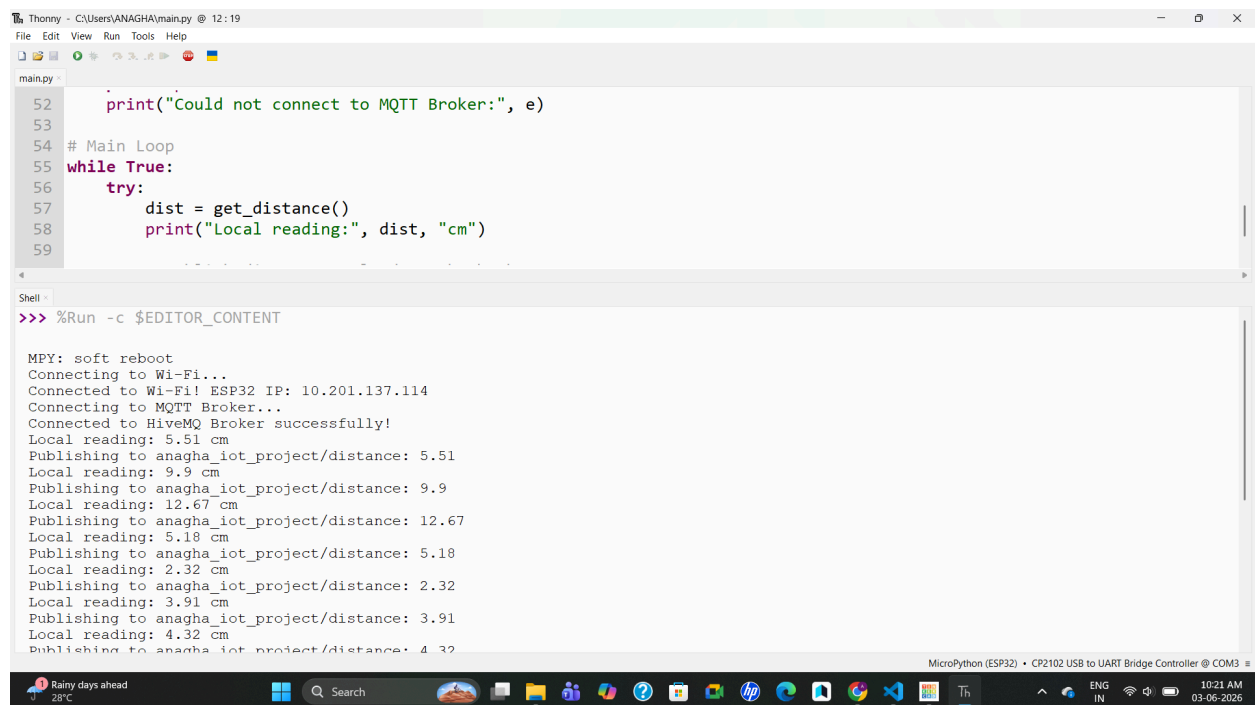
Step 4: Testing Your New MQTT Architecture

We are using a public cloud broker so we don't need to copy your laptop's IP address into Thonny this time. Both devices just look for broker.hivemq.com.

1. **Start VS Code First:** Run `mqtt_laptop.py`. It should say: [*] Successfully connected to HiveMQ Broker!... Waiting for ESP32 data...
2. **Start Thonny:** Run `script` on the ESP32.
3. **Check the outputs:** ESP32 will print Publishing to topic... and your VS Code terminal will instantaneously print out the MQTT Received - Distance: XX cm data

OUTPUT

THONNY



The screenshot shows the Thonny IDE interface. The top pane displays a Python script named `main.py` with the following code:

```
52     print("Could not connect to MQTT Broker:", e)
53
54 # Main Loop
55 while True:
56     try:
57         dist = get_distance()
58         print("Local reading:", dist, "cm")
59
```

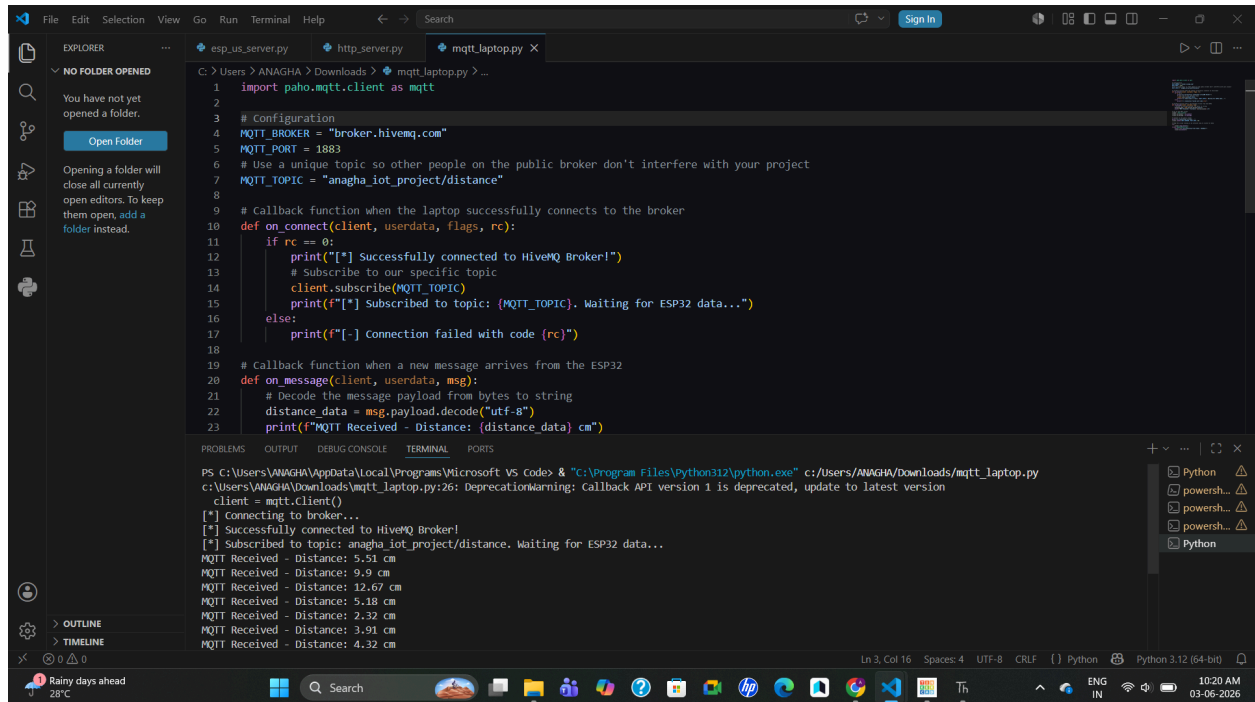
The bottom pane shows the execution output in a shell window. The output indicates a successful connection to the MQTT broker and subsequent data publishing and local printing:

```
>>> %Run -c $EDITOR_CONTENT

MPY: soft reboot
Connecting to Wi-Fi...
Connected to Wi-Fi! ESP32 IP: 10.201.137.114
Connecting to MQTT Broker...
Connected to HiveMQ Broker successfully!
Local reading: 5.51 cm
Publishing to anagha_iot_project/distance: 5.51
Local reading: 9.9 cm
Publishing to anagha_iot_project/distance: 9.9
Local reading: 12.67 cm
Publishing to anagha_iot_project/distance: 12.67
Local reading: 5.18 cm
Publishing to anagha_iot_project/distance: 5.18
Local reading: 2.32 cm
Publishing to anagha_iot_project/distance: 2.32
Local reading: 3.91 cm
Publishing to anagha_iot_project/distance: 3.91
Local reading: 4.32 cm
Publishing to anagha_iot_project/distance: 4.32
```

The status bar at the bottom indicates the environment is MicroPython (ESP32) running on a CP2102 USB to UART Bridge Controller @ COM3.

VS CODE



The screenshot displays the Visual Studio Code editor with a Python script named `mqtt_laptop.py` open. The script is configured to connect to a HiveMQ broker and subscribe to a specific topic. The terminal output shows the successful connection and the receipt of distance data from an ESP32 device.

```
1 import paho.mqtt.client as mqtt
2
3 # Configuration
4 MQTT_BROKER = "broker.hivemq.com"
5 MQTT_PORT = 1883
6 # Use a unique topic so other people on the public broker don't interfere with your project
7 MQTT_TOPIC = "anagha_iot_project/distance"
8
9 # Callback function when the laptop successfully connects to the broker
10 def on_connect(client, userdata, flags, rc):
11     if rc == 0:
12         print("[*] Successfully connected to HiveMQ Broker!")
13         # Subscribe to our specific topic
14         client.subscribe(MQTT_TOPIC)
15         print(f"[*] Subscribed to topic: {MQTT_TOPIC}. Waiting for ESP32 data...")
16     else:
17         print(f"[-] Connection failed with code {rc}")
18
19 # Callback function when a new message arrives from the ESP32
20 def on_message(client, userdata, msg):
21     # Decode the message payload from bytes to string
22     distance_data = msg.payload.decode("utf-8")
23     print(f"MQTT Received - Distance: {distance_data} cm")
```

The terminal output shows the following sequence of events:

```
PS C:\Users\ANAGHA\AppData\Local\Programs\Microsoft VS Code> & "C:\Program Files\Python312\python.exe" c:/Users/ANAGHA/Downloads/mqtt_laptop.py
c:\Users\ANAGHA\Downloads\mqtt_laptop.py:26: DeprecationWarning: Callback API version 1 is deprecated, update to latest version
client = mqtt.Client()
[*] connecting to broker...
[*] Successfully connected to HiveMQ Broker!
[*] Subscribed to topic: anagha_iot_project/distance. Waiting for ESP32 data...
MQTT Received - Distance: 5.51 cm
MQTT Received - Distance: 9.9 cm
MQTT Received - Distance: 12.67 cm
MQTT Received - Distance: 5.18 cm
MQTT Received - Distance: 2.32 cm
MQTT Received - Distance: 3.91 cm
MQTT Received - Distance: 4.32 cm
```